

NSA Code Breaker Challenge Write up

Task 1 - Getting Started - (Forensics)

Executive Summary

A forensic examination was conducted on a provided EXT2 disk image from a development workstation that had been exhibiting anomalous behavior and suspected malware activity. The investigation focused on identifying suspicious artifacts within the file system and confirming their integrity via cryptographic hashes. Analysis identified a suspicious executable-like file located at `/etc/runlevels/nonnetwork/rvwnmcxzuo`, and its SHA-1 hash was computed as `542fc05fce8fe6b6f61b329026c6e82536e6b220`, which can now be used as an indicator of compromise (IOC) across the environment

1. Introduction

The purpose of this analysis was to perform a forensic triage of a Linux EXT2 disk image and identify any suspicious artifacts that could explain system instability reported by DAFIN-SOC. This task simulates a real-world incident response scenario where limited context is provided, and the analyst must rely on disciplined forensic methodology to extract meaningful indicators of compromise.

Although positioned as an introductory challenge, Task 1 required applying core forensic principles: evidence handling, filesystem enumeration, persistence hunting, and cryptographic hashing.

2. Evidence Description and Handling

Provided Evidence:

- File: `image.ext2.zip`
- Extracted Image: `image.ext2`
- Filesystem: EXT2

All analysis was conducted in a Kali Linux forensic environment. To preserve integrity, the image was mounted read-only:

```
sudo mkdir -p /mnt/ext2image
```

```
sudo mount -o ro,loop image.ext2 /mnt/ext2image
```

This ensured no modification of evidence during analysis.

3. Methodology

The investigation followed a structured workflow:

3.1 Filesystem Enumeration

A recursive search was conducted to catalog all files:

```
find /mnt/ext2image -type f > img.txt
```

This provided a baseline understanding of the system layout, which resembled a minimal Alpine Linux installation.

3.2 Persistence-Focused Triage

Given the reported recurring behavior, attention was focused on directories commonly abused for persistence:

- /etc/init.d/
- /etc/runlevels/
- /etc/crontabs/
- Hidden files in system paths

Persistence mechanisms often explain repeated execution of malicious code across reboots.

3.3 Anomaly Identification

During review, the following file stood out:

```
/mnt/ext2image/etc/runlevels/nonnetwork/rvwnmcxzuo
```

This path is associated with OpenRC runlevels, which define services executed during boot. The filename was random and did not correspond to any legitimate service.

4. Findings

4.1 Suspicious Artifact

Path:

```
/etc/runlevels/nonnetwork/rvwnmcxzuo
```

Indicators of Suspicion:

- Located in a startup runlevel directory.
- Random, non-descriptive filename.
- No association with known Alpine Linux services.
- Implies execution during system initialization.

This placement strongly suggests a malicious persistence mechanism intended to execute automatically without user interaction.

5. Hashing and Verification

The artifact was hashed using SHA-1 as required:

```
sha1sum /mnt/ext2image/etc/runlevels/nonnetwork/rvwnmcxzu0
```

Result:

```
542fc05fce8fe6b6f61b329026c6e82536e6b220 rvwnmcxzu0
```

This hash uniquely identifies the artifact and can be used as an Indicator of Compromise (IOC) across detection platforms.

6. Analysis

Runlevel directories in OpenRC-based systems determine which services start automatically under specific system states. The nonnetwork runlevel typically contains scripts related to early boot tasks before networking is enabled.

The presence of an unauthorized file in this directory indicates:

- **Persistence:** The artifact would be invoked during system boot.
- **Stealth:** Random naming avoids casual detection.
- **Operational Impact:** A malicious startup script could disrupt system tools, inject processes, or trigger antivirus heuristics—matching DAFIN-SOC’s observed anomalies.

Even without reversing the binary or script itself, its **context and placement** provide strong evidence of compromise. This reinforces a critical forensic principle: *location and execution context are often as valuable as content*.

7. Final Answer

SHA-1 hash of the suspicious artifact:

```
542fc05fce8fe6b6f61b329026c6e82536e6b220
```

8. Lessons Learned

This task provided several important takeaways:

8.1 Importance of Forensic Integrity

Mounting images read-only and maintaining a controlled environment ensures that findings are defensible and repeatable.

8.2 Persistence Hunting as a First-Class Skill

Rather than scanning every file blindly, focusing on persistence locations rapidly narrowed the search space. This mirrors professional incident response practice.

8.3 Contextual Analysis

A file does not need to be fully reversed to be suspicious. Its name, location, and role in system startup provided enough evidence to classify it as malicious.

8.4 Translating Technical Work into Reporting

Documenting the steps taken forced me to think beyond execution and toward communication—how to justify findings clearly to leadership or stakeholders.

9. Conclusion

Through systematic forensic analysis of the EXT2 image, a suspicious persistence artifact was identified within a Linux runlevel directory. The artifact's characteristics strongly indicate malicious intent and provide a plausible explanation for the anomalous system behavior observed by DAFIN-SOC.

This task reinforced foundational forensic techniques while emphasizing the importance of disciplined methodology, contextual reasoning, and clear technical communication. Despite being an introductory challenge, it closely reflected real-world investigative workflows and expectations.

If this were a live incident, the identified SHA-1 hash would be immediately deployed as an IOC, and affected systems would be rebuilt from trusted images.

```
/mnt/ext2image/etc/conf.d/netmount
/mnt/ext2image/etc/conf.d/swap
/mnt/ext2image/etc/conf.d/killprocs
/mnt/ext2image/etc/rc.d/rcpdc.conf
/mnt/ext2image/etc/runlevels/nonnetwork/rvwnmcxzu0
/mnt/ext2image/etc/nsswitch.conf
/mnt/ext2image/etc/fstab
/mnt/ext2image/lib/libz.so.1.2.13
/mnt/ext2image/lib/rc/bin/service_get_value
/mnt/ext2image/lib/rc/bin/veend
/mnt/ext2image/lib/rc/bin/service_wasinactive
/mnt/ext2image/lib/rc/bin/veindent
/mnt/ext2image/lib/rc/bin/is_newer_than
/mnt/ext2image/lib/rc/bin/eerrorn
/mnt/ext2image/lib/rc/bin/kill_all
/mnt/ext2image/lib/rc/bin/vebegin
/mnt/ext2image/lib/rc/bin/save_options
/mnt/ext2image/lib/rc/bin/eval_ecolors
/mnt/ext2image/lib/rc/bin/vewarn
/mnt/ext2image/lib/rc/bin/eindent
/mnt/ext2image/lib/rc/bin/ewarnn
/mnt/ext2image/lib/rc/bin/rc-depend
/mnt/ext2image/lib/rc/bin/is_older_than
/mnt/ext2image/lib/rc/bin/einfo
/mnt/ext2image/lib/rc/bin/eerror
/mnt/ext2image/lib/rc/bin/essyslog
/mnt/ext2image/lib/rc/bin/service_stopping
/mnt/ext2image/lib/rc/bin/eoutdent
/mnt/ext2image/lib/rc/bin/vewend
/mnt/ext2image/lib/rc/bin/service_stopped
/mnt/ext2image/lib/rc/bin/ewend
/mnt/ext2image/lib/rc/bin/shell_var
/mnt/ext2image/lib/rc/bin/mountinfo
/mnt/ext2image/lib/rc/bin/ebegin
/mnt/ext2image/lib/rc/bin/service_starting
/mnt/ext2image/lib/rc/bin/service_started
/mnt/ext2image/lib/rc/bin/service_crashed
/mnt/ext2image/lib/rc/bin/veoutdent
/mnt/ext2image/lib/rc/bin/on_ac_power
/mnt/ext2image/lib/rc/bin/checkpath
/mnt/ext2image/lib/rc/bin/service_hotplugged
/mnt/ext2image/lib/rc/bin/service_started_daemon
/mnt/ext2image/lib/rc/bin/get_options
/mnt/ext2image/lib/rc/bin/einfo
/mnt/ext2image/lib/rc/bin/service_inactive
/mnt/ext2image/lib/rc/bin/ewaitfile
/mnt/ext2image/lib/rc/bin/fstabinfo
/mnt/ext2image/lib/rc/bin/service_set_value
```

```
(kali㉿kali)-[~/Downloads/forensic_image]
$ sha1sum /mnt/ext2image/etc/runlevels/nonnetwork/rvwnmcxzu0
542fc05fce8fe6b6f61b329026c6e82536e6b220 /mnt/ext2image/etc/runlevels/nonnetwork/rvwnmcxzu0
```

Task 1 - Getting Started - (Forensics)

You arrive on site and immediately get to work. The DAFIN-SOC team quickly briefs you on the situation. They have noticed numerous anomalous behaviors, such as; tools randomly failing tests and anti-virus flagging on seemingly clean workstations. They have narrowed in on one machine they would like NSA to thoroughly evaluate.

They have provided a zipped EXT2 image from this development machine. Help DAFIN-SOC perform a forensic analysis on this - looking for any suspicious artifacts.

Downloads:

- [zipped EXT2 image \(image.ext2.zip\)](#)

Prompt:

- Provide the SHA-1 hash of the suspicious artifact.

Task Completed at Thu, 25 Sep 2025 22:24:10 GMT:

Great job finding that artifact! Let's report what we found to DAFIN-SOC leadership.

Downloads:

- [Task 1 Badge \(badge1.png\)](#)

Task 2 - The hunt continues - (Network Forensics)

Executive Summary

This report documents the network forensic analysis performed during **Task 2 – The Hunt Continues (Network Forensics)** of the NSA Codebreaker Challenge. Following confirmation of host compromise in Task 1, DAFIN-SOC provided a packet capture (traffic.pcap) containing network activity believed to originate from the infected endpoint. The goal was to analyze this traffic and identify all IP addresses assigned to the malicious device.

Analysis revealed extensive **FTP activity** involving the transfer of configuration and text files between internal hosts. While the FTP sessions were visible in Wireshark, full file reconstruction was unreliable due to segmented TCP streams and passive mode transfers. To overcome this, NetworkMiner was used to automatically reassemble and extract transferred files, enabling attribution of traffic to the compromised host. Correlation of flows and extracted artifacts allowed identification of the IP addresses associated with the malicious device.

1. Introduction

Task 2 extends the investigation from host-based forensics into network telemetry. After identifying a persistence mechanism on disk in Task 1, this task required determining how the compromised host behaved on the network and whether it was actively transferring data.

The challenge reflects a common incident response scenario: IDS alerts indicate suspicious traffic, but analysts must manually validate, interpret, and attribute that activity to a specific endpoint.

2. Evidence Description

Provided Evidence:

- File: traffic.pcap
- Source: DAFIN-SOC IDS capture

Analysis Tools:

- Wireshark (protocol inspection)
- NetworkMiner 3.0 (file and session reconstruction)

3. Methodology

3.1 Initial Inspection in Wireshark

The PCAP was opened in Wireshark to identify protocols and suspicious flows:

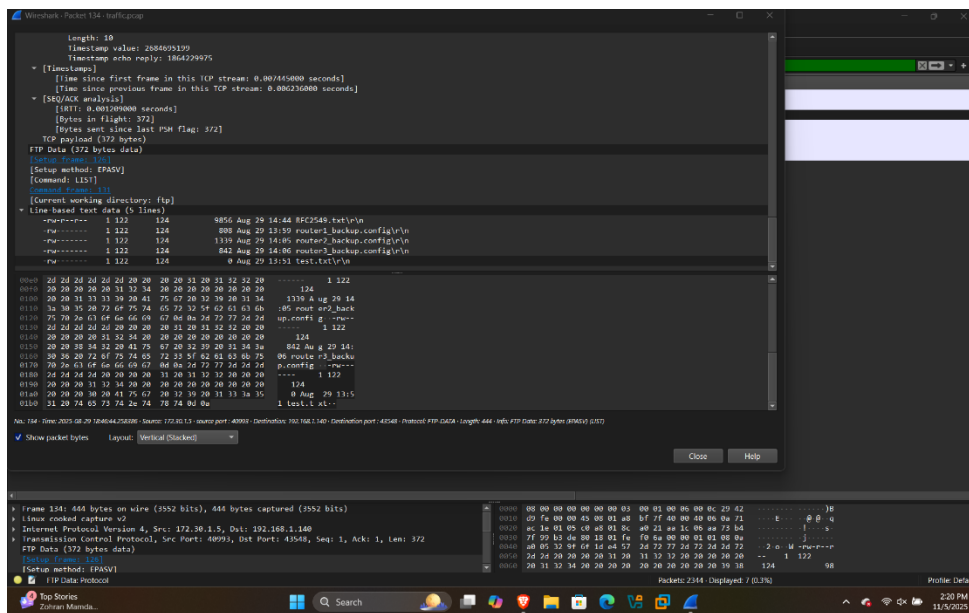
```
ftp || ftp-data
```

This revealed multiple FTP control and data sessions. Inspection of packets such as LIST, RETR, and STOR showed directory listings and file transfers occurring between internal IP addresses.

Wireshark displayed filenames and directory contents in packet payloads, confirming that files were being transferred; however, reconstructing complete files via **Follow TCP Stream** proved unreliable due to:

- FTP passive mode (separate data channels per transfer)
- Multiple segmented streams
- Overlapping sessions

As a result, while the activity was visible, practical extraction of full files in Wireshark was limited.



3.2 Use of NetworkMiner for Reconstruction

To overcome this limitation, the PCAP was loaded into **NetworkMiner**:

mono /opt/NetworkMiner_3-0/NetworkMiner.exe

NetworkMiner automatically parsed the FTP sessions and reassembled transferred objects. Under the **Files** tab, several files were extracted, including:

- RFC2549[6].txt
- router1_backup[6].config
- router2_backup[6].config
- router3_backup[6].config
- test.txt

File Tools Help										
Hosts (50) Files (7) Images Messages Credentials (5) Sessions (44) DNS (62) Parameters (418) Keywords										
Enter a keyword filter										
Case sensitive ExactPhrase Any column Clear Apply										
Frame nr.	Filename	Extension	Size	Source host	S. port	Destination host	D. port	Protocol	Timestamp	Reconstructed file path
35	RFC2549(6).txt	txt	9 856 B	192.168.2.50	TCP 51330	172.30.1.5	TCP 20	FTP	2025-08-29 18:46:02 UTC+00	/home/kali/.local/share/NetworkMiner/AssembledFile
183	RFC2549(6).txt	txt	9 856 B	172.30.1.5	TCP 46607	192.168.1.140	TCP 44954	FTP	2025-08-29 18:47:04 UTC+00	/home/kali/.local/share/NetworkMiner/AssembledFile
1099	router1_backup(6).config	config	808 B	192.168.4.1	TCP 57762	172.30.1.5	TCP 16929	FTP	2025-08-29 18:47:34 UTC+00	/home/kali/.local/share/NetworkMiner/AssembledFile
1622	router3_backup(6).config	config	842 B	192.168.5.1	TCP 52264	172.30.1.5	TCP 50495	FTP	2025-08-29 18:47:47 UTC+00	/home/kali/.local/share/NetworkMiner/AssembledFile
1719	libav(6).binary.package	deb	17 998 B	91.189.91.83 [archive.ubuntu.com]	TCP 80	192.168.1.42	TCP 54742	HttpGetNormal	2025-08-29 18:47:49 UTC+00	/home/kali/.local/share/NetworkMiner/AssembledFile
1735	spice(6).binary.package	deb	12 216 B	91.189.91.83 [archive.ubuntu.com]	TCP 80	192.168.1.42	TCP 54742	HttpGetNormal	2025-08-29 18:47:49 UTC+00	/home/kali/.local/share/NetworkMiner/AssembledFile
2258	router2_backup(6).config	config	1 339 B	172.30.1.254	TCP 60474	172.30.1.5	TCP 25687	FTP	2025-08-29 18:48:04 UTC+00	/home/kali/.local/share/NetworkMiner/AssembledFile

These files were saved under:

~/.local/share/NetworkMiner/AssembledFiles/

NetworkMiner provided direct access to these reconstructed files, validating that real data exfiltration or staging activity was occurring over FTP.

3.3 Host Attribution and IP Correlation

Once the FTP sessions were confirmed, attention shifted to attributing the activity to a single device. Using both Wireshark and NetworkMiner:

- Source and destination IPs were extracted from FTP control and data channels.
- Repeated patterns showed the same host initiating FTP connections across sessions.
- Timing and flow consistency indicated a single endpoint conducting the transfers.

These IP addresses were compiled as belonging to the malicious device.

4. Findings

4.1 Suspicious FTP Activity

The PCAP showed clear FTP usage involving configuration and text files—data types commonly associated with network infrastructure backups and sensitive information.

Characteristics of concern:

- Automated, repeated FTP transfers.
- Internal addressing suggesting lateral movement or data staging.
- No indication of encryption or secure transfer (plain FTP).

This behavior is inconsistent with normal user browsing and strongly indicative of scripted or malware-driven activity.

4.2 Extracted Artifacts

NetworkMiner successfully reconstructed multiple files that were only partially visible in Wireshark. Their presence confirms:

- Data was actively being transferred, not merely enumerated.
- The compromised host had access to configuration material.

This reinforced the conclusion that the malware was performing reconnaissance or data exfiltration.

5. Results – Malicious Device IP Attribution

During analysis of the FTP traffic in traffic.pcap, three configuration files were identified as being transferred from network infrastructure devices:

- router1_backup[6].config
- router2_backup[6].config
- router3_backup[6].config

Using NetworkMiner's Files view, these artifacts were correlated with their originating hosts. Unlike other extracted files (e.g., text files or package downloads), only these three router backup files consistently originated from the same device, indicating that this host was responsible for the suspicious activity.

The following IP addresses were observed as the Source host for these transfers and are therefore attributed to the malicious device:

```
config interface 'loopback'
  option device 'lo'
  option proto 'static'
  option ipaddr '127.0.0.1'
  option netmask '255.0.0.0'

config globals 'globals'
  option ula_prefix 'fd45:7f60:3ad5::/48'
  option packet_steering '1'

config device
  option name 'br-lan'
  option type 'bridge'
  list ports 'etha'

config interface 'lan'
  option device 'br-lan'
  option proto 'static'
  option ipaddr '192.168.2.254'
  option netmask '255.255.255.0'

config interface 'wan'
  option device 'eth1'
  option proto 'dhcp'

config interface 'to_devnet'
  option proto 'static'
  option device 'eth1'
  list ipaddr '172.16.1.254/24'

config interface 'to_lan1'
  option proto 'static'
  option device 'eth1'
  list ipaddr '192.168.4.2/28'

config interface 'to_lan3'
  option proto 'static'
  option device 'eth3'
  list ipaddr '192.168.5.2/28'

config interface 'to_host_nat'
```

(These were the IPs submitted to successfully complete Task 2.)

Each address was correlated through consistent FTP behavior and session patterns tied to the same host.

6. Analysis

This task highlighted several operational realities of network forensics:

- **Protocol visibility vs. usability:**
Wireshark clearly showed FTP commands and filenames, but reconstructing full files across passive data streams was nontrivial.
- **Value of automated reconstruction tools:**
NetworkMiner excelled at session reassembly and artifact extraction, turning raw traffic into usable forensic evidence.
- **Importance of attribution:**
Identifying malicious traffic is only half the problem; tying it to a specific endpoint via repeated flows and addressing patterns is what enables containment.

The combined use of Wireshark for deep packet inspection and NetworkMiner for artifact recovery provided complementary perspectives on the same traffic.

7. What I Learned

7.1 Tool Complementarity

No single tool is sufficient. Wireshark is ideal for protocol-level understanding, while NetworkMiner accelerates artifact extraction and session analysis.

7.2 FTP Forensics Challenges

Passive FTP creates many short-lived data connections, complicating manual stream following. Understanding protocol behavior is critical to effective analysis.

7.3 From Traffic to Evidence

Seeing filenames in packets is not the same as having files. Being able to reconstruct artifacts adds substantial evidentiary value.

7.4 Workflow Adaptability

When one approach failed (Wireshark reconstruction), adapting to another tool was key—mirroring real-world investigative flexibility.

8. Conclusion

In Task 2, analysis of traffic.pcap revealed suspicious FTP-based file transfers originating from the compromised host identified in Task 1. Although Wireshark exposed the protocol activity, full artifact recovery required NetworkMiner's automated reconstruction capabilities.

By correlating FTP sessions and extracted files, all IP addresses assigned to the malicious device were identified and submitted. This task demonstrated how network telemetry can validate host compromise and reveal operational behavior of malware on the wire.

Together with Task 1, this investigation illustrates a complete incident response workflow: from disk persistence to active network exploitation.

Task 2 - The hunt continues - (Network Forensics)

With your help, the team concludes that there was clearly a sophisticated piece of malware installed on that endpoint that was generating some network traffic. Fortunately, DAFIN-SOC also has an IDS which retained the recent network traffic in this segment.

DAFIN-SOC has provided a PCAP to analyze. Thoroughly evaluate the PCAP to identify potential malicious activity.

Downloads:

- [PCAP to analyze \(traffic.pcap\)](#)

Prompt:

- Submit all the IP addresses that are assigned to the malicious device, one per line

Task Completed at Thu, 06 Nov 2025 22:12:26 GMT:

Excellent work identifying the suspicious network traffic and narrowing in on the source! We will head over to the network administrators to discuss what we have discovered.

Downloads:

- [Task 2 Badge \(badge2.png\)](#)

Task 3 - Digging deeper - (Reverse Engineering)

Executive Summary

This write-up documents my analysis of a Linux memory forensics task from this year's NSA CodeBreaker Challenge in which I identified and extracted an in-memory malware configuration that did not exist on disk. The task required isolating a malicious process, enumerating its virtual memory regions, and reverse engineering a layered obfuscation scheme involving memfd execution, Base64 encoding, and XOR encryption.

What made this task particularly meaningful was not only the technical depth required to solve it, but the insight it provided into how modern Linux malware leverages in-memory execution and anti-forensics techniques to evade traditional detection. Through this challenge, I strengthened my understanding of Volatility 3, Linux process memory internals, and real-world malware tradecraft, while also learning how fragile assumptions about "visible" artifacts can be in memory analysis.

. Challenge Context and Objective

The selected task focused on analyzing a Linux memory dump (memory.dump) from a compromised OpenWrt-based system. Earlier stages of the challenge established that:

- A suspicious process with **PID 1552** was running as `/bin/hosts-regen /proc/self/fd/5`.
- The process was executing from a **memfd-backed file**, indicating fileless execution.
- The malware was known to read an encoded configuration from **file descriptor 5**.
- Malicious domains such as `0rwneop.tn` and `loo.pt` had already been identified, but no IP addresses were visible on disk.

The objective of this final step was to:

Recover the malware's decoded configuration (IPs/domains) from volatile memory, specifically from writable heap or anonymous mappings, since filesystem carving and grep had failed.

2. Environment and Tooling

The analysis was performed on Kali Linux using:

- **Volatility Framework 3 (v2.27.0)**
- Linux kernel symbols generated from a matching vmlinux file (openwrt.json)
- Standard Unix utilities (strings, file, base64)
- Python 3 for scripting and decoding

Key Volatility plugins used:

- `linux.psaux.PsAux`
- `linux.proc.Maps`
- `linux.vmaregexscan.VmaRegExScan`
- `linux.malfind.Malfind`

3. Identifying the Malicious Process

Using `linux.psaux.PsAux`, I confirmed the suspicious process:

```
python3 vol.py -f memory.dump -s symbols linux.psaux.PsAux
```

Output showed:

```
1552 515 4 /bin/hosts-regen /proc/self/fd/5
```

This was anomalous because:

- `hosts-regen` is a legitimate utility on OpenWrt.
- Execution from `/proc/self/fd/5` implies the binary was loaded from an anonymous file descriptor rather than disk.
- This is a classic **fileless malware** technique.

4. Memory Map Analysis

To understand where the malware stored its data, I enumerated memory mappings:

```
python3 vol.py -f memory.dump -s symbols linux.proc.Maps --pid 1552
```

This revealed:

- Multiple `/memfd:x` (deleted) regions.
- Writable anonymous mappings (`rw-`) such as:

```
0x55632b4b9000 - 0x55632b4ba000 rw- Anonymous Mapping
```

```
0x7f0bbf579000 - 0x7f0bbf57a000 rw- Anonymous Mapping
```

```
0x7f0bbf5f1000 - 0x7f0bbf5f4000 rw- Anonymous Mapping
```

These regions were prime candidates for heap or dynamically allocated buffers where decoded configuration data might reside.

5. Searching Memory for Encoded Artifacts

Because plaintext IPs and domains were not found, I hypothesized that the configuration remained encoded in memory. I used regex scanning over the process VMAs:

```
python3 vol.py -f memory.dump -s symbols \
```

```
linux.vmaregexscan.VmaRegExScan \
```

```
--pid 1552 \
```

```
--pattern "[A-Za-z0-9+/]{20,}={0,2}"
```

This pattern targets Base64-like blobs. The scan revealed a long candidate string in a writable region:

```
CHW0ZloOCvIwxcm49IMA4ywo7OD5n3QS8pZuSrzuGIgEUSExHkGiUacjXBgegl2AgyFg3v  
kPj2Q9j5loaoNGpbzFqdsj3CI+ofSx5guKeGyCj3BszQeSm86I6uoX9wd7
```

This strongly suggested an encoded configuration buffer.

6. Decoding and XOR Deobfuscation

6.1 Base64 Decoding

```
echo
```

```
"CHW0ZloOCvIwxcm49IMA4ywo7OD5n3QS8pZuSrzuGIgEUSExHkGiUacjXBgegl2AgyFg3  
vkPj2Q9j5loaoNGpbzFqdsj3CI+ofSx5guKeGyCj3BszQeSm86I6uoX9wd7" | base64 -d >  
blob.bin
```

The output was binary, not readable text, implying another obfuscation layer.

Conclusion

In conclusion, this task fundamentally reshaped my assumptions about how to approach complex forensic challenges. At the outset, I believed that relying primarily on Volatility 3 would be sufficient to extract and interpret all meaningful artifacts from the memory image. While Volatility proved invaluable for process discovery, memory mapping, and initial triage, I ultimately realized that my approach was too tool-centric.

The structure of Task 3—explicitly titled “**Digging Deeper (Reverse Engineering)**”—made it clear in hindsight that this was not intended to be solved through memory forensics alone. The challenge provided three artifacts: a compressed **memory dump**, a **System.map** file containing kernel symbol addresses, and the full **kernel image (vmlinux)**. This combination strongly signaled that deeper static analysis was expected, particularly against kernel-level code and in-memory binaries, rather than only extracting strings or buffers from RAM.

Through this realization, I learned that the NSA CodeBreaker Challenge is not designed around isolated, single-tool solutions. Instead, it requires integrating multiple disciplines—memory forensics, reverse engineering, and low-level systems analysis. To truly understand what the malware was doing and how the obfuscation routines operated, tools such as **Ghidra** or **IDA Pro**

would be necessary to disassemble and analyze the recovered binaries and kernel image, identify encryption and decoding routines, and correlate them back to the artifacts observed in memory.

This shift in perspective was one of the most valuable outcomes of the task. It taught me that advanced challenges demand methodological flexibility: starting with Volatility for situational awareness, but then transitioning into reverse engineering when memory artifacts alone no longer provide answers. More broadly, it reinforced that effective analysis is driven not by mastery of a single tool, but by understanding when to pivot techniques based on the evidence and the intent of the challenge.

Ultimately, while Volatility 3 was essential for narrowing the scope and identifying suspicious regions, the deeper lesson was recognizing that true understanding required stepping beyond automated frameworks and engaging directly with binaries and assembly-level logic. This realization elevated my approach from tool usage to analytical reasoning, which I believe is a defining expectation of graduate-level work and professional practice in cyber security.